

Ejercicio #1 – Análisis de Sentimientos

Alumna: María Emilia Peña Onganía

Introducción

El análisis de sentimiento es una de las aplicaciones más utilizadas dentro del Procesamiento de Lenguaje Natural (NLP). Permite identificar automáticamente la opinión o emoción expresada en un texto, clasificándola generalmente como positiva, negativa o neutral.

En este trabajo se comparan dos enfoques diferentes para la clasificación de sentimientos utilizando reseñas del dataset Yelp Reviews:

- Método basado en léxicos.
- Método basado en un modelo preentrenado (TextBlob).

El objetivo es evaluar el desempeño de ambos métodos mediante métricas de clasificación y analizar sus ventajas y limitaciones.

Dataset utilizado

Se utilizó el dataset Yelp Reviews disponible públicamente en Kaggle.

Las reseñas contienen una puntuación de estrellas otorgada por los usuarios. Para construir la variable objetivo se definió:

- Sentimiento positivo (1): reseñas con 4 o 5 estrellas.
 - Sentimiento negativo (0): reseñas con menos de 4 estrellas.
-

Metodología

Método basado en léxicos

Este enfoque utiliza una lista predefinida de palabras positivas y negativas. La clasificación se realiza contando la cantidad de términos positivos y negativos presentes en cada reseña.

Si predominan las palabras positivas, la reseña se clasifica como positiva; en caso contrario, como negativa.

Código utilizado

```
positive_words = [  
    "good", "great", "excellent", "amazing",
```

```

        "love", "best", "nice", "perfect",

        "wonderful", "awesome", "fantastic"

    ]

negative_words = [

    "bad", "terrible", "worst", "awful",

    "poor", "hate", "horrible",

    "disappointing", "boring", "slow"

]

def lexicon_sentiment(text):

    text = str(text).lower()

    pos = sum(word in text for word in positive_words)

    neg = sum(word in text for word in negative_words)

    if pos > neg:

        return 1

    else:

        return 0

df["lexicon_prediction"] = df["text"].apply(

    lexicon_sentiment

)

df[["text", "sentiment", "lexicon_prediction"]].head()

```

```

... /tmp/ipykernel_15393/2985179494.py:29: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
df["lexicon_prediction"] = df["text"].apply(

```

	text	sentiment	lexicon_prediction
0	My wife took me here on my birthday for breakf...	1	1
1	I have no idea why some people give bad review...	1	1
2	love the gyro plate. Rice is so good and I als...	1	1
3	Rosie, Dakota, and I LOVE Chaparral Dog Park!!...	1	1
4	General Manager Scott Petello is a good egg!!!...	1	1

Método basado en TextBlob

TextBlob es una biblioteca de procesamiento de lenguaje natural que incorpora modelos preentrenados para calcular automáticamente la polaridad de un texto.

Cada reseña recibe una puntuación de polaridad. Los textos con polaridad positiva son clasificados como positivos y el resto como negativos.

Código utilizado

```
!pip install textblob

from textblob import TextBlob

def textblob_sentiment(text):

    polarity = TextBlob(str(text)).sentiment.polarity

    if polarity > 0:
        return 1
    else:
        return 0

df["textblob_prediction"] = df["text"].apply(
    textblob_sentiment
)
```

```
Requirement already satisfied: textblob in /usr/local/lib/python3.12/dist-packages (0.15.0)
Requirement already satisfied: nltk<3.9 in /usr/local/lib/python3.12/dist-packages (from textblob) (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (8.4.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (1.5.3)
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (4.67.3)
/tmp/ipykernel_15393/2754575397.py:17: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
df["textblob_prediction"] = df["text"].apply(
```

Métricas de evaluación

Para comparar ambos enfoques se utilizaron las siguientes métricas:

- Accuracy
- Precision
- Recall
- F1 Score

Estas métricas permiten evaluar tanto la exactitud general del modelo como su capacidad para identificar correctamente las distintas clases.

Resultados

Método	Accuracy	Precision	Recall	F1 Score
Léxico	0.726	0.769	0.851	0.812
TextBlob	0.735	0.731	0.970	0.834

Código utilizado

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

print("==== METODO LEXICO ====")
```

```

print("Accuracy:",
      accuracy_score(df["sentiment"],
                     df["lexicon_prediction"]))

print("Precision:",
      precision_score(df["sentiment"],
                      df["lexicon_prediction"]))

print("Recall:",
      recall_score(df["sentiment"],
                   df["lexicon_prediction"]))

print("F1:",
      f1_score(df["sentiment"],
               df["lexicon_prediction"]))

print("==== TEXTBLOB ====")
print("Accuracy:",
      accuracy_score(df["sentiment"],
                     df["textblob_prediction"]))

print("Precision:",
      precision_score(df["sentiment"],
                      df["textblob_prediction"]))

print("Recall:",
      recall_score(df["sentiment"],
                   df["textblob_prediction"]))

print("F1:",
      f1_score(df["sentiment"],
               df["textblob_prediction"]))

```

```

==== METODO LEXICO ====
Accuracy: 0.7263
Precision: 0.7691183574223845
Recall: 0.8598995191687169
F1: 0.8116181438242962

```

```

*** ===== TEXTBLOB =====
Accuracy: 0.7346
Precision: 0.7311875286976852
Recall: 0.9698382631582258

```

Nota metodológica:

Para simplificar el ejercicio, la evaluación se realizó utilizando el conjunto completo de datos disponible. En un escenario productivo, se recomienda dividir el dataset en conjuntos de entrenamiento y validación (por ejemplo, 80%-20%) para obtener una estimación más robusta del desempeño de los modelos y evitar posibles sesgos en la evaluación.

Análisis de resultados

Los resultados muestran que ambos enfoques logran un desempeño aceptable en la tarea de clasificación.

El método basado en léxicos presenta una implementación sencilla y rápida, aunque depende fuertemente de la calidad del diccionario utilizado y no logra capturar adecuadamente el contexto del lenguaje.

Por otro lado, TextBlob obtiene mejores resultados generales, especialmente en Recall y F1 Score. Esto indica una mayor capacidad para identificar correctamente las reseñas positivas y una mejor adaptación a variaciones lingüísticas presentes en los textos.

Ventajas y limitaciones

Método Léxico

Ventajas:

- Fácil implementación.
- No requiere entrenamiento.
- Bajo costo computacional.

Limitaciones:

- Sensible a la selección de palabras.
- No interpreta contexto ni sarcasmo.
- Menor capacidad de generalización.

TextBlob

Ventajas:

- Utiliza conocimiento lingüístico preentrenado.
- Mejor comprensión semántica.
- Mayor desempeño general.

Limitaciones:

- Dependencia de librerías externas.
- Mayor complejidad computacional.
- Posibles errores en dominios específicos.

Conclusión

A partir de los resultados obtenidos se concluye que el modelo basado en TextBlob ofrece un mejor rendimiento para la clasificación automática de sentimientos en reseñas de Yelp.

Si bien los enfoques léxicos continúan siendo útiles por su simplicidad y rapidez de implementación, los modelos preentrenados representan una alternativa más robusta para aplicaciones reales de análisis de sentimiento.

Como trabajo futuro podrían evaluarse modelos más avanzados basados en Transformers, como BERT o RoBERTa, para obtener mejoras adicionales en la calidad de clasificación.